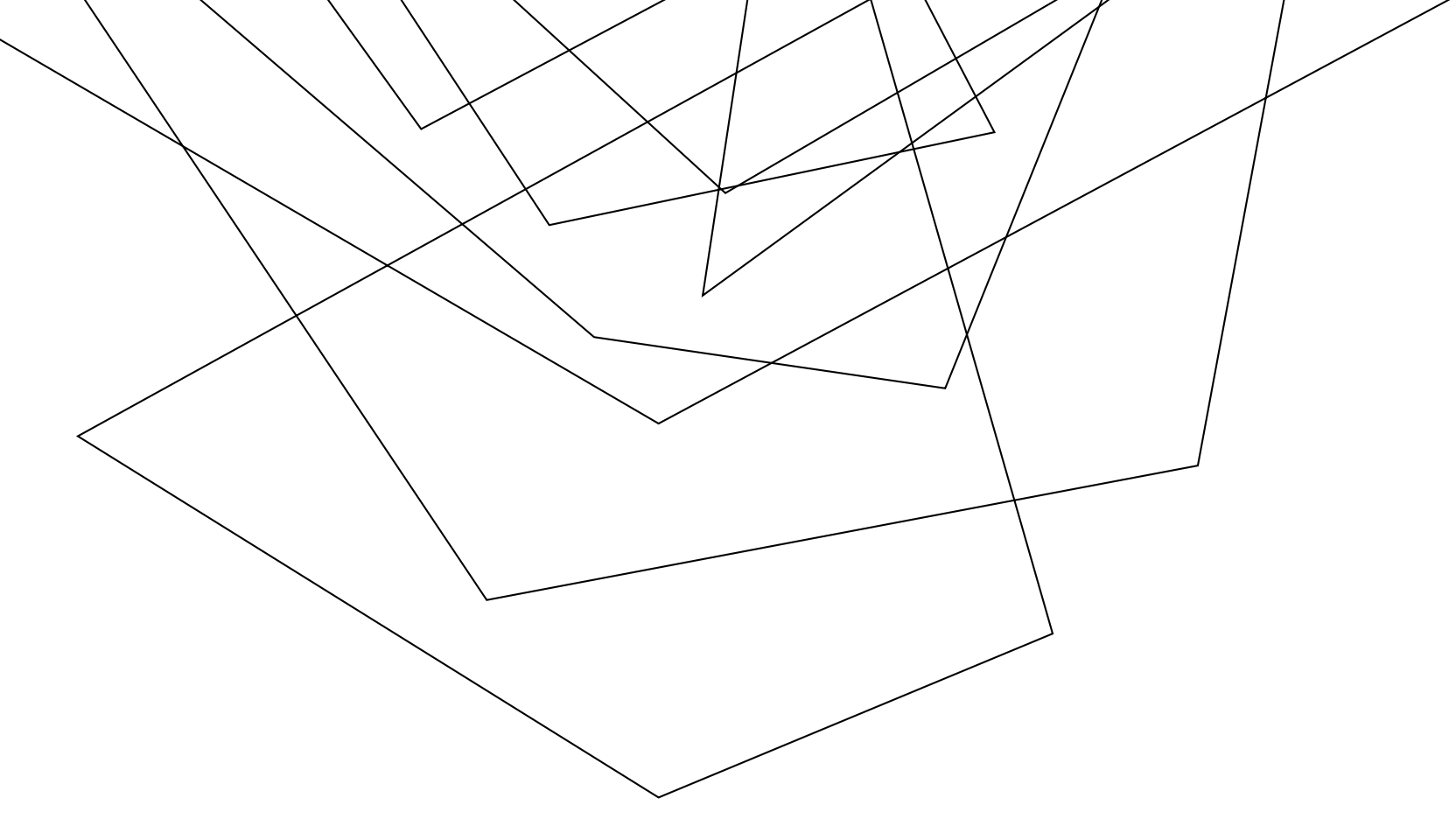




**Hof
University**
University of
Applied Sciences



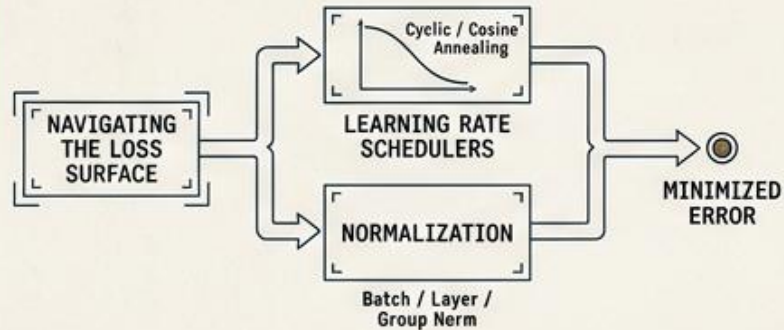
APPLIED DEEP LEARNING

08 – EFFICIENT LEARNING

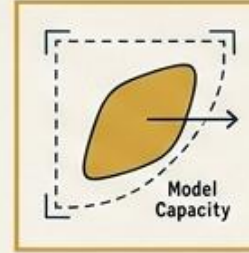
PROF. DR. LARS ACKERMANN-IGL

TAXONOMY

OPTIMIZATION DYNAMICS

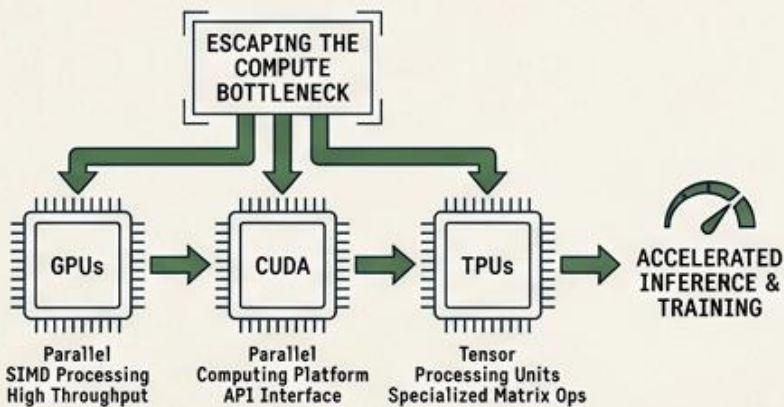
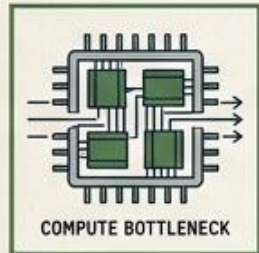


GENERALIZATION STRATEGIES

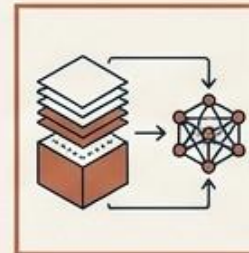


CONSTRAINING CAPACITY	TECHNIQUES
Prevents Overfitting Improves Robustness	REGULARIZATION (L1, L2) $-\lambda \sum w $ $-\lambda \sum w^2$
	DROPOUT
	LABEL SMOOTHING

HARDWARE ACCELERATION



REPRESENTATION EFFICIENCY



MODEL DISTILLATION	 Teacher Student	Teacher-Student Transfer Soft Labels
EMBEDDINGS	 Dimensionality Reduction Dense Vectors	
SUPERPOSITION	 Interference Patterns Shared Weights	

COMPACT & EFFICIENT MODELS



OPTIMIZATION DYNAMICS

- Learning rate schedulers
- Normalization

LEARNING RATE SCHEDULERS

So far

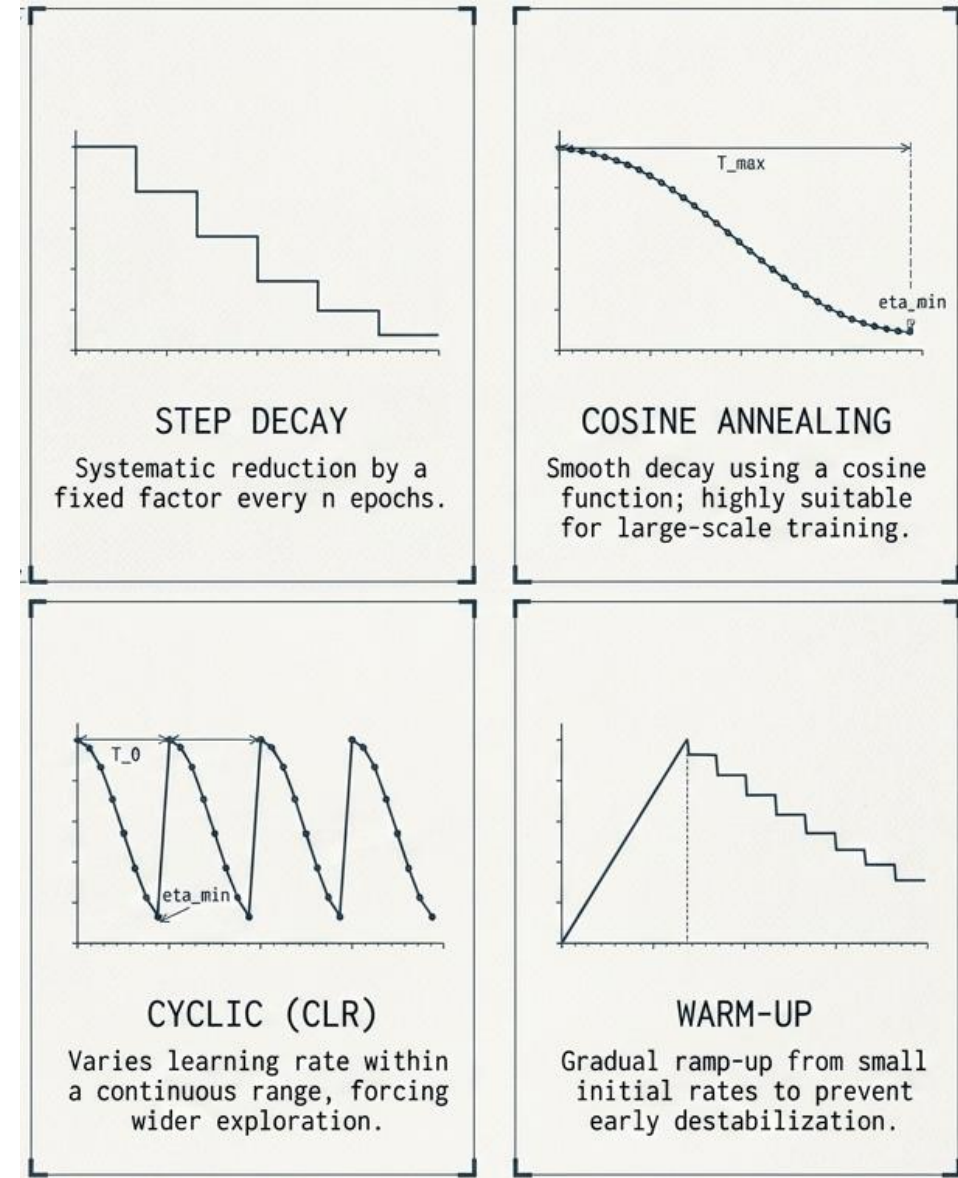
We assumed that the learning rate remained constant throughout a training run.

Change introduced now

Optimizer schedulers dynamically adjust the learning rate during training.

Goal

Improve convergence, escape local minima, and ensure generalization.



LEARNING RATE SCHEDULERS – IMPLEMENTATION

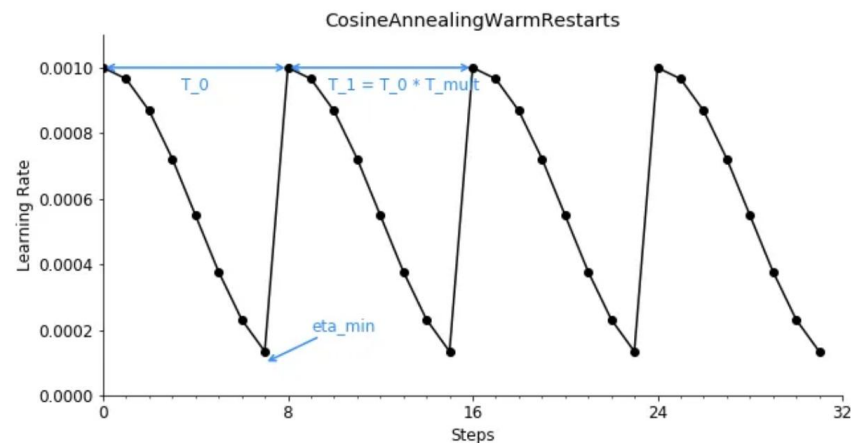
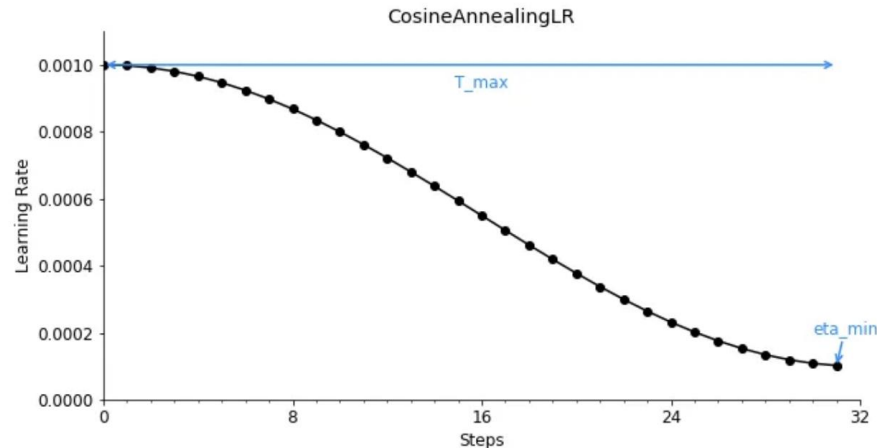


Complete notebook available (soon)
in our Moodle course!

Keras

```
reduce_lr = keras.callbacks.ReduceLROnPlateau(  
    monitor="val_loss",  
    factor=0.2,  
    patience=2,  
    verbose=1,  
    min_lr=1e-5  
)  
  
history_plateau = train(callbacks=[reduce_lr])
```

```
cosine = keras.optimizers.schedules.CosineDecayRestarts(  
    initial_learning_rate=INITIAL_LR,  
    first_decay_steps=5 * (len(x_train)//128),  
    t_mul=2.0,  
    m_mul=1.0,  
    alpha=0.001  
)  
  
history_cosine = train(scheduler=cosine)
```

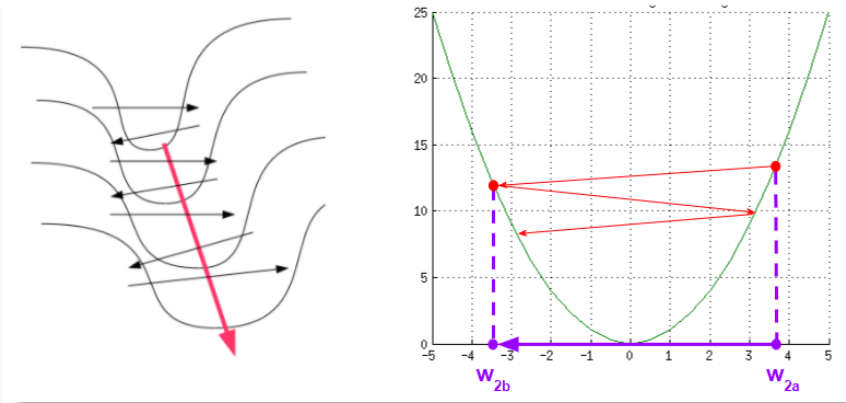
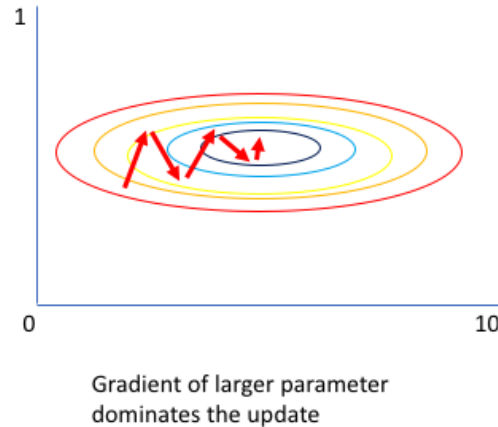


NORMALIZATION: WHY IT IS NECESSARY

Problem

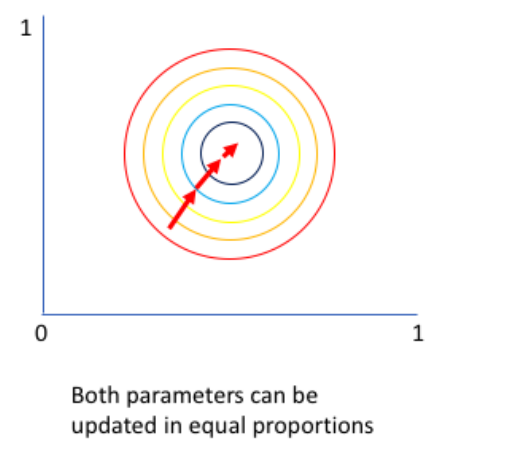
Unevenly scaled data can lead to problematic optimizer behavior as the error surface is elongated and, thus, form ravines/canyons.

- Large-scale features have steep slopes on the error surface.
- Small-scale features have flat slopes.



Goal

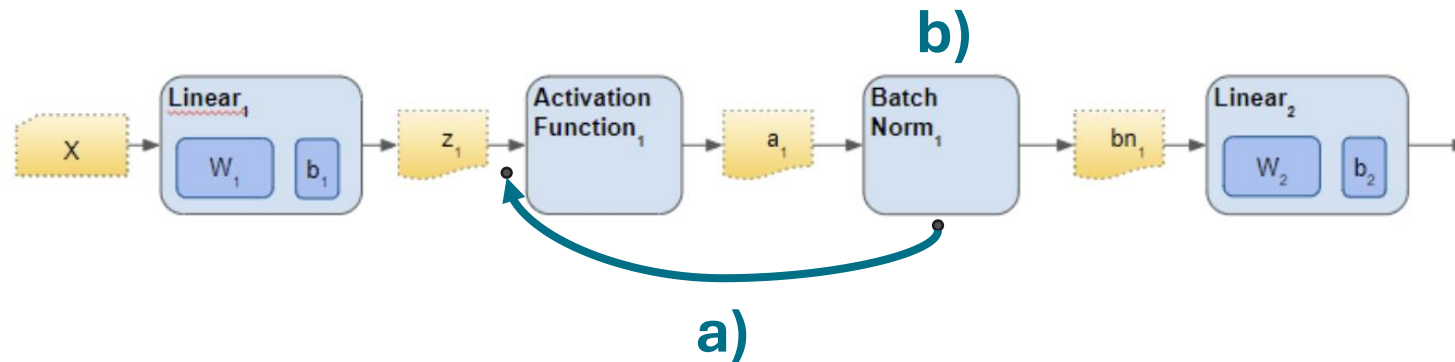
Smooth navigation of the error surface.



BATCH NORMALIZATION

Key concept

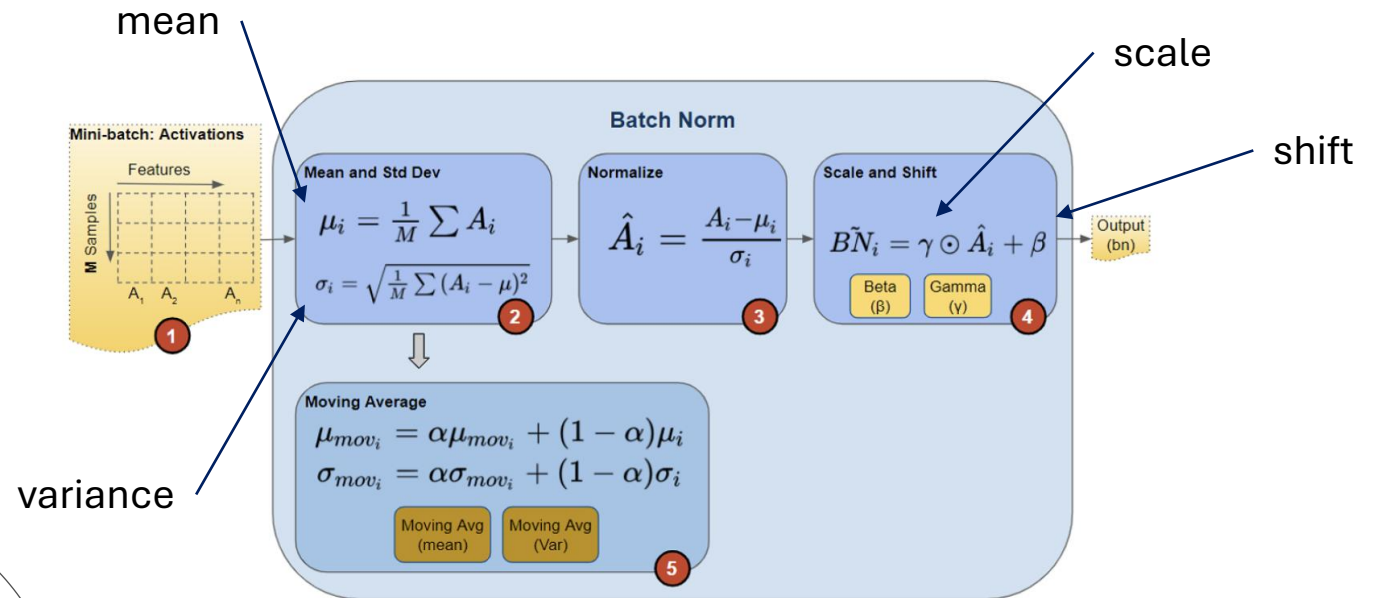
- Batch norm is another layer inserted into network architecture
- Normalizes output ...
 - a) ... before calculating the activation or
 - b) ... before passing to next layer or (preferred)



BATCH NORMALIZATION – TRAINING

Details

- For standardization 2 stored (exponential) moving averages values
 - μ_{mov_i} : Continuously updated average
 - σ_{mov_i} : Continuously updated standard deviation
 - Rationale: The net evolves batch by batch and, thus, “older” batches are less representative than more recent batches. Thus, let the mean/variance statistics history fade out (using a rather small α !).
- During training we normalize using batch-dependent mean and variance!
- Learn 2 learnable parameters:
 - β (shift): Moving the mean.
 - γ (scale): Scaling the variance.
 - Rationale: Scale the values so that it uses the characteristics of the following activation function (e.g., distribute values so that more or less values are set to 0 by a ReLU)



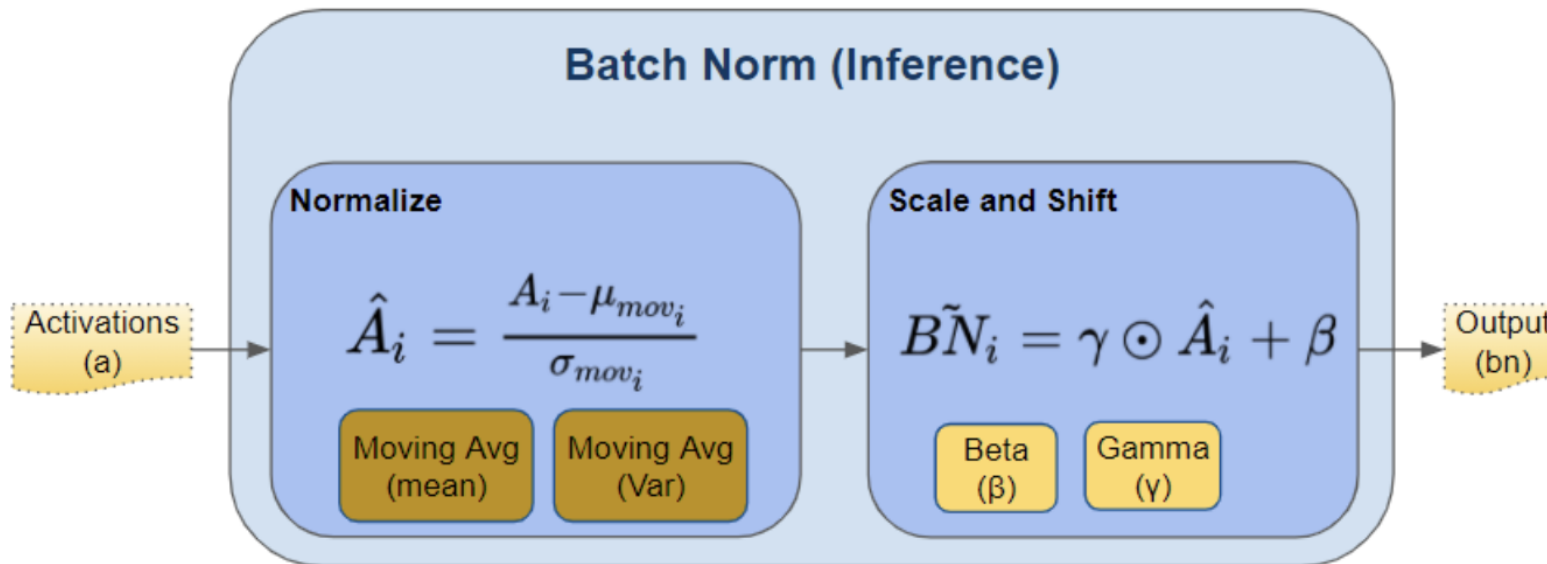
(Note: The picture assumes that you apply it to the activations; however, the computational principle is the same for variant a. But remember: It is usually placed after activation function)

Why do we need that? Next slide!

BATCH NORMALIZATION – INFERENCE

Why we need the moving averages

- During inference, we only have a single sample, not a mini batch → mean and variance?
- Moving averages used as good proxies



BATCH NORMALIZATION – HANDS-ON

- Remember: Batch normalization can be implemented in different ways, i.e.,
 - a) Adding it BEFORE applying activation functions or
 - b) Adding it AFTER applying activation functions
 - ... in the Jupyter notebook we exclusively rely on option a!



Complete notebook available (soon)
in our Moodle course!

```
model = keras.Sequential([  
    layers.Input(shape=(784,)),  
  
    layers.Dense(256, use_bias=False),  
    layers.BatchNormalization(),  
    layers.Activation("relu"),  
  
    layers.Dense(128, use_bias=False),  
    layers.BatchNormalization(),  
    layers.Activation("relu"),  
  
    layers.Dense(64, use_bias=False),  
    layers.BatchNormalization(),  
    layers.Activation("relu"),  
  
    layers.Dense(10, activation="softmax")  
)  
  
model.summary()
```

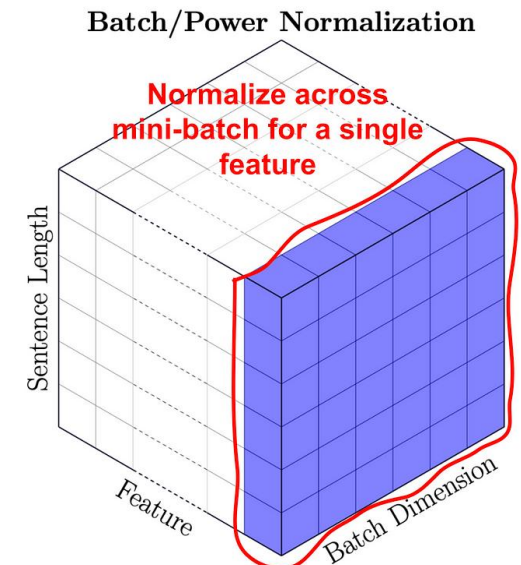
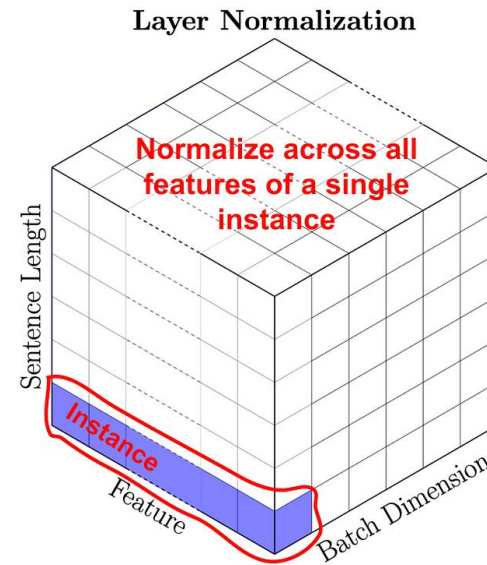
LAYER NORMALIZATION

Core principle

Calculates the mean and variance of the activations for each individual input sample, considering all features

Where are the differences?

- Batch normalization
 - Good in case features are related, like computer vision (fully connected, CNNs)
- Layer normalization
 - Good in case batch sizes are too small, batches are likely to be noisy
 - Reduces internal covariance shift
 - Good with RNNs and Transformers
 - Same operations in train and inference





GENERALIZATION STRATEGIES

- Penalizing extrema -> Regularization
- Changing the training
- Changing the architecture

REGULARIZATION

Problem

A flexible neural network can:

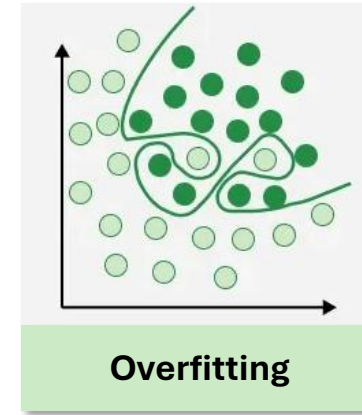
- memorize training data
- learn noise instead of patterns
- perform poorly on new (unseen) data

Core idea for a solution

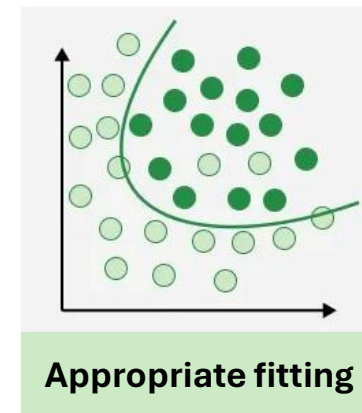
Change the learning goal from “**minimize training error**” to “**minimize training error + limit complexity**”!

Variants we discuss

- L1 regularization
- Dropout
- Early stopping



↓ Regularization



L1 REGULARIZATION

Principle

- add penalty term to loss which depends on weights
- Scaling factor α : models how expensive complexity is
 - Small α : model complexity is cheap
 - High α : model complexity is expensive

$$Loss = Error(y, \hat{y}) + \alpha \sum_{i=1}^N |w_i|$$

← Loss ... combination of result error and high weight penalty

$$w_{\text{new}} = w - \left(\lambda \frac{\partial \text{Error}}{\partial w} + \lambda \frac{\partial (\alpha \sum_{i=1}^N |w_i|)}{\partial w} \right)$$

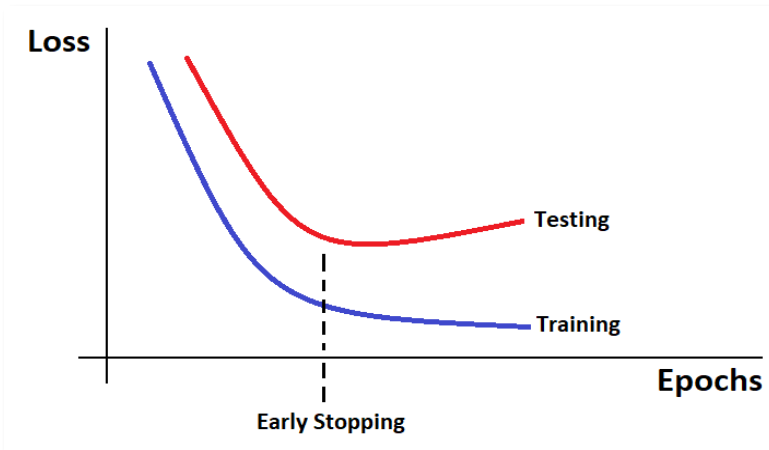
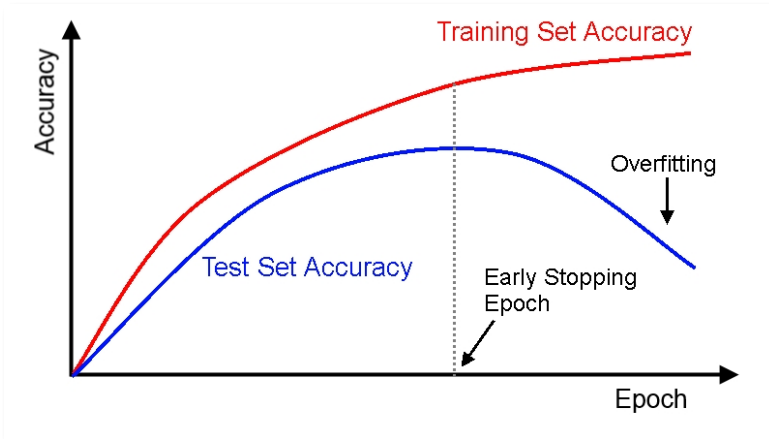
← Weight update ... change weight using a gradient that considers both

$$\frac{\partial (\alpha \sum_{i=1}^N |w_i|)}{\partial w} = \begin{cases} \alpha & w > 0 \\ -\alpha & w < 0 \end{cases}$$

← Makes sure to scale positive and negative weights in the right direction

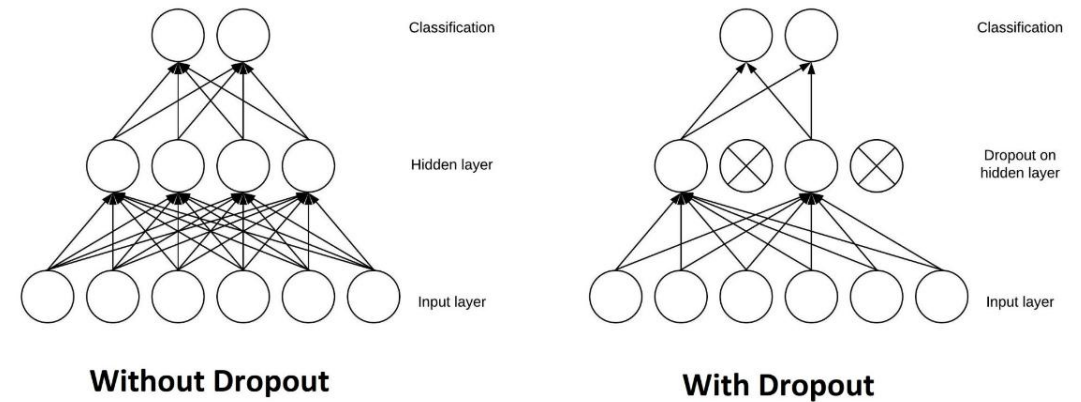
EARLY STOPPING AND DROPOUT (AT A GLANCE)

Early stopping



Dropout

- Randomly dropping neurons in a layer (no update on backprop)
- Creates redundancies in the network, reduces complexity





HARDWARE ACCELERATION

- CUDA
- TPUs

CUDA

Compute Unified Device Architecture

- Parallel computing platform by NVIDIA.
- Deep integration with PyTorch & TensorFlow
- Enables highly parallel execution of tensor/matrix operations using GPU cores.
- Keras uses CUDA indirectly through TensorFlow, which calls CUDA/cuDNN when a GPU is available.

Example: Check for GPU

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print(tf.config.list_physical_devices('GPU')) # CUDA GPU check

model = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy')

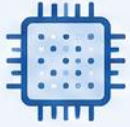
model.fit(x_train, y_train, epochs=5)
```

Example: Explicit device placement

```
with tf.device('/GPU:0'): # or '/CPU:0'
    model.fit(x_train, y_train)
```

TENSOR PROCESSING UNITS – TPUs

1. What is a TPU?



A TPU is a custom-designed chip by Google, built specifically for machine learning. It performs the core math operations of neural networks extremely efficiently.

2. What TPUs Do (Key Strengths)



High Throughput for Matrix Operations
TPUs are optimized for large matrix multiplications, the backbone of neural network training.



Massive Parallelism
They perform the same operation on many data elements at the same time.



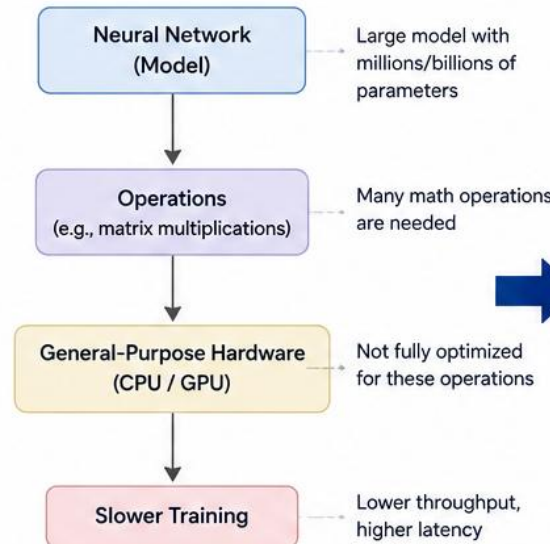
High Memory Bandwidth
TPUs have high-bandwidth memory (HBM) to feed data to compute units quickly, reducing bottlenecks.



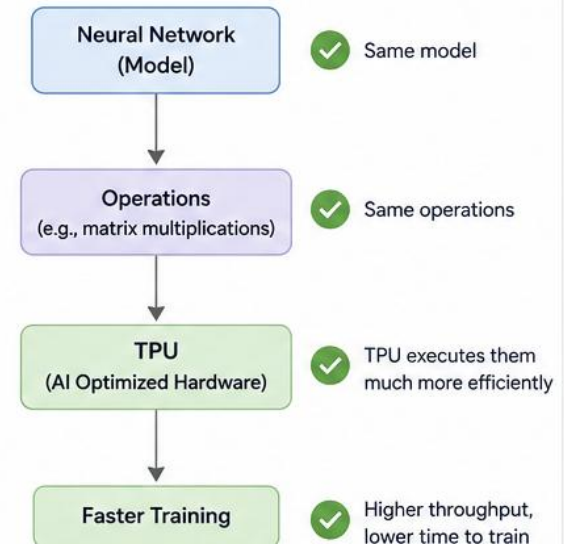
Energy Efficiency
TPUs deliver more performance per watt compared to CPUs and GPUs.

3. How TPUs Speed Up Neural Network Training

Typical CPU / GPU



With TPU



4. Bottom Line



TPUs accelerate neural network training by performing the right operations in parallel, with high memory bandwidth and energy efficiency—leading to faster training, lower cost, and the ability to train larger models.



Faster Training

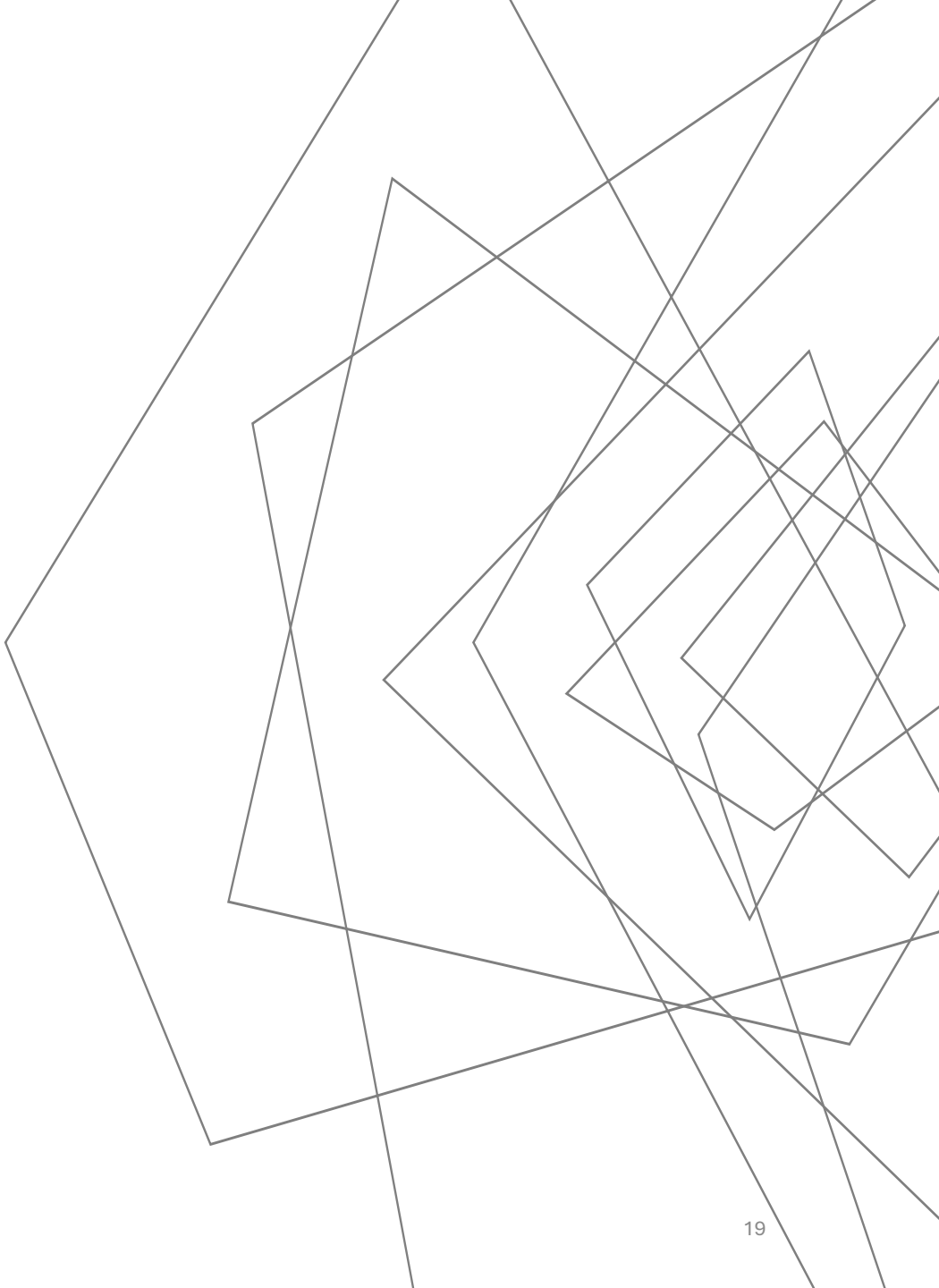


Lower Cost



Train Larger Models

TPUs are used in Google Cloud TPUs and supports frameworks like TensorFlow / Keras, JAX, and PyTorch (via XLA).



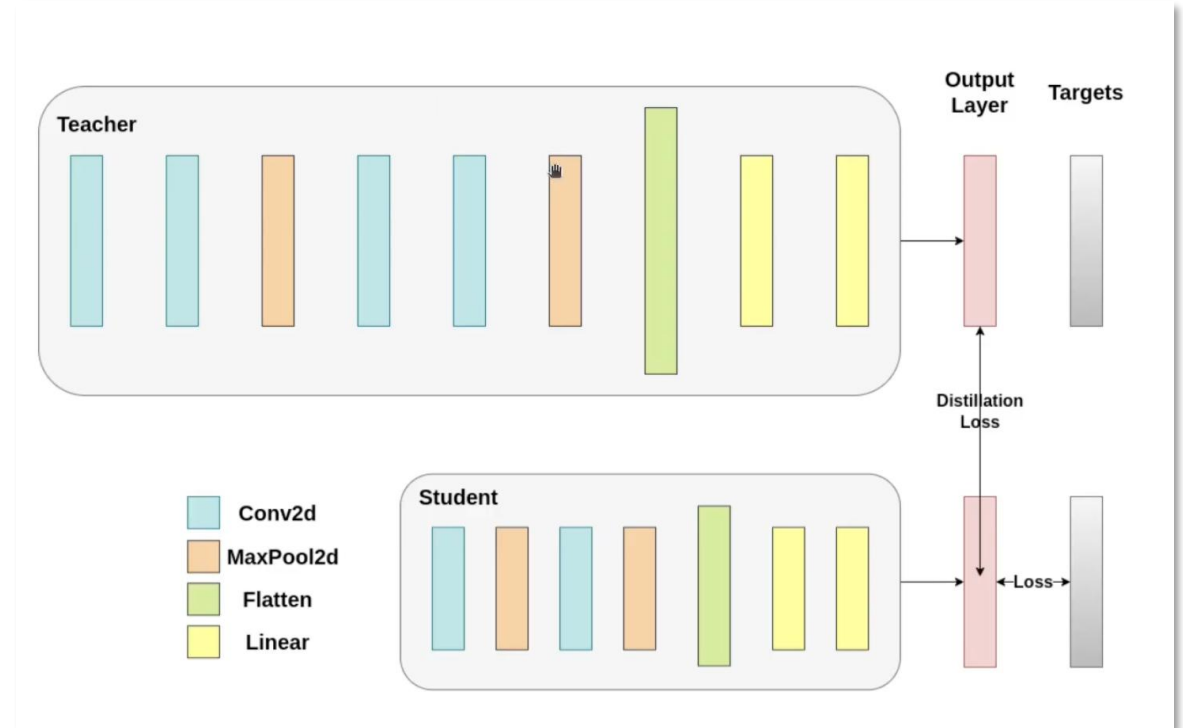
REPRESENTATION EFFICIENCY

- Model/Knowledge distillation
- Quantization
- ... many more possibilities

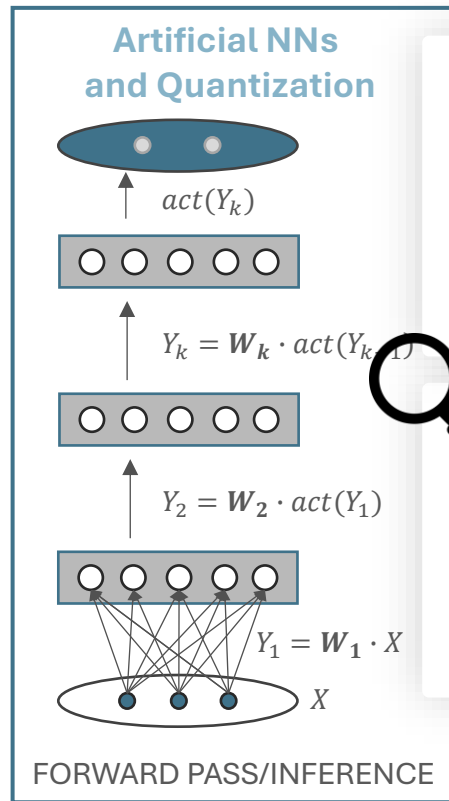
MODEL DISTILLATION

Principle

- Also knowledge distillation
- Abilities of larger model (teacher) are conveyed to a smaller model (pupil)
- Implementations
 - Using GANs to make pupil indistinguishable from teacher
 - Cross-Modal distillation: one works in images, one in text
 - Soft target dist.: using logits instead of hard outputs



QUANTIZATION



Definitions

X ... Input

Y_k ... Net input of layer k

W_k ... Weight matrix of layer k

$act(Y_k)$... Activation of layer k

Propagation ($Y = W \cdot X$)

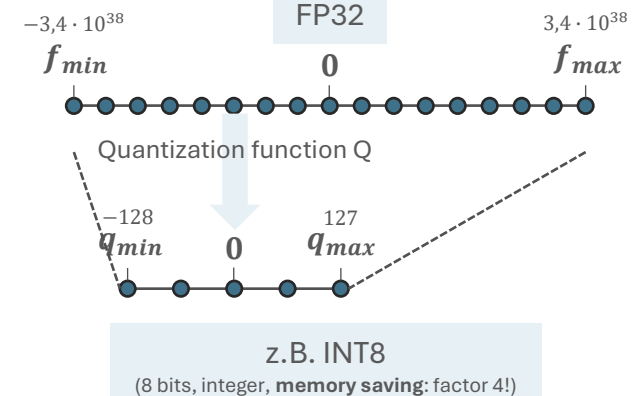
$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} w_{00} & w_{01} & w_{02} \\ w_{10} & w_{11} & w_{12} \\ w_{20} & w_{21} & w_{22} \\ w_{30} & w_{31} & w_{32} \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

The root cause! Floating-Point Values!
(FP32 – 32 Bit, 7 decimals)



The weights usually do not use the entire FP32 value range! Can we use that insight somehow?

Quantization



- a) Post-training Quantization (PTQ):** **accuracy**
Static quantization after training
- b) Quantization-aware Training (QAT):**
Quantization during training or as a calibration step

QUESTIONS?





DONE.

95028 Hof
Alfons-Goppel-Platz 1
Phone +49 9281 409-3000
Fax +49 9281 409-4000
mail@hof-university.de
www.hof-university.de